```
"""
commands.py — All slash commands for RFRXDex
"""

import discord
from discord import app_commands
from discord.ext import commands
import math
from typing import Optional

import database as db
import card_utils as cu

# ────────────────────────────────────────────────────────────────────────────
# Helper: require card lookup
# ────────────────────────────────────────────────────────────────────────────

def embed_error(msg: str) -> discord.Embed:
    return discord.Embed(description=f"❌ {msg}", color=0xFF4444)

def embed_ok(title: str, desc: str, color: int = 0x00CC66) -> discord.Embed:
    return discord.Embed(title=title, description=desc, color=color)


# ────────────────────────────────────────────────────────────────────────────
# COG: Collection & Inventory
# ────────────────────────────────────────────────────────────────────────────

class CollectionCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    @app_commands.command(name="collection", description="View your card collecti
    @app_commands.describe(page="Page number", user="View another user's collecti
    async def collection(self, interaction: discord.Interaction, page: int = 1, u
        target = user or interaction.user
        db.ensure_user(str(target.id), target.display_name)

        inventory = db.get_user_inventory(str(target.id))
        info_cards = db.get_user_info_cards(str(target.id))
        user_data = db.get_user(str(target.id))

        if not inventory and not info_cards:
            await interaction.response.send_message(
```

```
            embed=embed_error(f"**{target.display_name}** has no cards yet!")
        )
        return

    per_page = 8
    items, total_pages = cu.paginate(inventory, page, per_page)
    page = max(1, min(page, total_pages))

    embed = discord.Embed(
        title=f"📇 {target.display_name}'s Collection",
        color=0x5865F2,
        description=f"Total cards: **{len(inventory)}** | Info cards: **{len(
    )

    for inv in items:
        card = cu.get_card_by_id(inv["card_id"])
        if not card:
            continue
        v_emoji = cu.get_variant_emoji(inv["variant"])
        r_emoji = cu.get_rarity_emoji(card["rarity"])
        current_val = db.get_current_value(card["id"], inv["variant"], card["
        embed.add_field(
            name=f"{v_emoji} {card['name']} [{inv['variant']}]",
            value=f"{r_emoji} {card['rarity'].capitalize()} • 🪙 {current_val
            inline=True
        )

    if info_cards and page == 1:
        info_names = []
        for ic in info_cards[:5]:
            ic_card = cu.get_card_by_id(ic["card_id"])
            if ic_card:
                info_names.append(f"📋 {ic_card['name']}")
        if info_names:
            embed.add_field(name="📋 Info Cards", value="\n".join(info_names)

    embed.set_footer(text=f"Page {page}/{total_pages} • RFRXDex")
    embed.set_thumbnail(url=target.display_avatar.url)
    await interaction.response.send_message(embed=embed)

@app_commands.command(name="card", description="View details of a specific ca
@app_commands.describe(name="Card name or ID")
async def card_info(self, interaction: discord.Interaction, name: str):
    card = cu.get_card_by_name(name) or cu.get_card_by_id(name.lower().replac
    if not card:
        await interaction.response.send_message(embed=embed_error(f"Card `{na
        return
```

```
        color = cu.get_rarity_color(card["rarity"])
        r_emoji = cu.get_rarity_emoji(card["rarity"])
        current_val = db.get_current_value(card["id"], "Standard", card["base_val

        embed = discord.Embed(title=f"{r_emoji} {card['name']}", color=color)
        embed.set_image(url=card["image_url"])
        embed.add_field(name="📁 Type", value=card["type"].capitalize(), inline=T
        embed.add_field(name=f"{r_emoji} Rarity", value=card["rarity"].capitalize
        embed.add_field(name="🌕 Current Value", value=f"{current_val:,}", inline
        embed.add_field(name="🛒 Marketable", value="Yes" if card["marketable"] e
        embed.add_field(name="🎁 Giftable", value="Yes" if card["giftable"] else
        embed.add_field(name="🌀 Spawnable", value="Yes" if card["spawnable"] els
        if card.get("special_variants"):
            embed.add_field(name="✨ Variants", value=", ".join(card["special_var
        if card.get("info_card_id"):
            ic = cu.get_card_by_id(card["info_card_id"])
            if ic:
                embed.add_field(name="📋 Info Card", value=ic["name"], inline=Tru
        embed.set_footer(text="RFRXDex")
        await interaction.response.send_message(embed=embed)


# ────────────────────────────────────────────────────────────────────────────
# COG: Market System
# ────────────────────────────────────────────────────────────────────────────

class MarketCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    @app_commands.command(name="market", description="Browse the global card mark
    @app_commands.describe(card="Filter by card name (optional)")
    async def market(self, interaction: discord.Interaction, card: Optional[str]
        card_filter = None
        if card:
            card_filter = cu.get_card_by_name(card)
            if not card_filter:
                await interaction.response.send_message(embed=embed_error(f"Card
                return

        listings = db.get_active_listings(card_filter["id"] if card_filter else N

        if not listings:
            await interaction.response.send_message(
                embed=embed_ok("🏪 Global Market", "No cards listed for sale righ
                ephemeral=True
```

```
        )
        return

    embed = discord.Embed(
        title="🏪 RFRXDex Global Market",
        color=0xF1C40F,
        description=f"Showing {min(15, len(listings))} of {len(listings)} lis
    )

    for lst in listings[:15]:
        c = cu.get_card_by_id(lst["card_id"])
        if not c:
            continue
        v_emoji = cu.get_variant_emoji(lst["variant"])
        r_emoji = cu.get_rarity_emoji(c["rarity"])
        seller = interaction.guild.get_member(int(lst["seller_id"])) if inter
        seller_name = seller.display_name if seller else f"User#{lst['seller_
        embed.add_field(
            name=f"{v_emoji} {c['name']} [{lst['variant']}]",
            value=(
                f"{r_emoji} {c['rarity'].capitalize()} • 🪙 **{lst['price']:,
                f"Seller: {seller_name} • ID: `{lst['listing_id']}`"
            ),
            inline=True
        )
    embed.set_footer(text="Use /buy <listing_id> to purchase • RFRXDex")
    await interaction.response.send_message(embed=embed)

@app_commands.command(name="sell", description="List a card for sale on the m
@app_commands.describe(instance_id="Card instance ID (8-char from /collection
async def sell(self, interaction: discord.Interaction, instance_id: str, pric
    db.ensure_user(str(interaction.user.id), interaction.user.display_name)

    # Find full instance
    inv = db.get_inventory_instance(instance_id)
    if not inv:
        # Try partial match
        full_inv = db.get_user_inventory(str(interaction.user.id))
        matches = [i for i in full_inv if i["instance_id"].startswith(instanc
        if not matches:
            await interaction.response.send_message(embed=embed_error("Card n
            return
        inv = matches[0]

    if inv["user_id"] != str(interaction.user.id):
        await interaction.response.send_message(embed=embed_error("That card
        return
```

```
    card = cu.get_card_by_id(inv["card_id"])
    if not card:
        await interaction.response.send_message(embed=embed_error("Invalid ca
        return

    if not card["marketable"]:
        await interaction.response.send_message(embed=embed_error(f"**{card['
        return

    if price < 1:
        await interaction.response.send_message(embed=embed_error("Price must
        return

    # Remove from inventory and create listing
    db.remove_card_from_inventory(inv["instance_id"])
    lid = db.create_listing(str(interaction.user.id), card["id"], inv["varian

    v_emoji = cu.get_variant_emoji(inv["variant"])
    embed = discord.Embed(
        title="✅ Card Listed!",
        description=f"{v_emoji} **{cu.get_card_display_name(card, inv['varian
        color=0x00CC66
    )
    await interaction.response.send_message(embed=embed)

@app_commands.command(name="buy", description="Buy a card from the market")
@app_commands.describe(listing_id="The listing ID from /market")
async def buy(self, interaction: discord.Interaction, listing_id: int):
    db.ensure_user(str(interaction.user.id), interaction.user.display_name)

    listings = db.get_active_listings()
    listing = next((l for l in listings if l["listing_id"] == listing_id), No
    if not listing:
        await interaction.response.send_message(embed=embed_error("Listing no
        return

    if listing["seller_id"] == str(interaction.user.id):
        await interaction.response.send_message(embed=embed_error("You can't
        return

    buyer_data = db.get_user(str(interaction.user.id))
    if buyer_data["coins"] < listing["price"]:
        await interaction.response.send_message(
            embed=embed_error(f"Not enough coins! You need 🪙 {listing['price
            ephemeral=True
        )
```

```
        return

    # Complete transaction
    completed = db.complete_listing(listing_id, str(interaction.user.id))
    if not completed:
        await interaction.response.send_message(embed=embed_error("This listi
        return

    db.deduct_coins(str(interaction.user.id), listing["price"])
    db.add_coins(listing["seller_id"], listing["price"])

    # Give card to buyer
    new_instance = cu.generate_instance_id()
    db.add_card_to_inventory(
        str(interaction.user.id), listing["card_id"], listing["variant"],
        listing["price"], new_instance
    )

    card = cu.get_card_by_id(listing["card_id"])
    # Update dynamic price
    if card:
        db.update_dynamic_price(card["id"], listing["variant"], card["base_va
        db.record_sale(card["id"], listing["variant"], listing["price"], list

    v_emoji = cu.get_variant_emoji(listing["variant"])
    embed = discord.Embed(
        title="🎉 Purchase Complete!",
        description=(
            f"You bought {v_emoji} **{cu.get_card_display_name(card, listing[
            f"for **🪙 {listing['price']:,}**!"
        ),
        color=0x00CC66
    )
    await interaction.response.send_message(embed=embed)

@app_commands.command(name="delist", description="Remove your listing from th
@app_commands.describe(listing_id="The listing ID to cancel")
async def delist(self, interaction: discord.Interaction, listing_id: int):
    db.ensure_user(str(interaction.user.id), interaction.user.display_name)
    listing = next((l for l in db.get_active_listings() if l["listing_id"] ==
    if not listing:
        await interaction.response.send_message(embed=embed_error("Listing no
        return
    if listing["seller_id"] != str(interaction.user.id):
        await interaction.response.send_message(embed=embed_error("That's not
        return
```

```
        cancelled = db.cancel_listing(listing_id, str(interaction.user.id))
        if cancelled:
            # Return card to seller
            new_instance = cu.generate_instance_id()
            db.add_card_to_inventory(str(interaction.user.id), listing["card_id"]
            await interaction.response.send_message(embed=embed_ok("✅ Listing Ca
        else:
            await interaction.response.send_message(embed=embed_error("Could not


# ────────────────────────────────────────────────────────────────────────────
# COG: Trading
# ────────────────────────────────────────────────────────────────────────────

class TradeConfirmView(discord.ui.View):
    def __init__(self, trade_id: int, target_id: int):
        super().__init__(timeout=120)
        self.trade_id = trade_id
        self.target_id = target_id

    @discord.ui.button(label="✅ Accept", style=discord.ButtonStyle.success)
    async def accept(self, interaction: discord.Interaction, button: discord.ui.B
        if interaction.user.id != self.target_id:
            await interaction.response.send_message("This trade isn't for you!",
            return
        trade = db.get_trade(self.trade_id)
        if not trade or trade["status"] != "pending":
            await interaction.response.send_message("Trade is no longer active.",
            return

        # Execute trade: swap cards
        for inst_id in trade["offer_cards"]:
            inv = db.get_inventory_instance(inst_id)
            if inv:
                db.transfer_card(inst_id, str(self.target_id))
        for inst_id in trade["request_cards"]:
            inv = db.get_inventory_instance(inst_id)
            if inv:
                db.transfer_card(inst_id, trade["initiator_id"])

        db.resolve_trade(self.trade_id, "completed")
        self.stop()
        for child in self.children:
            child.disabled = True
        await interaction.message.edit(view=self)
        await interaction.response.send_message(f"✅ Trade `#{self.trade_id}` com
```

```
    @discord.ui.button(label="❌ Decline", style=discord.ButtonStyle.danger)
    async def decline(self, interaction: discord.Interaction, button: discord.ui.
        if interaction.user.id != self.target_id and str(interaction.user.id) !=
            await interaction.response.send_message("You can't decline this trade
            return
        db.resolve_trade(self.trade_id, "declined")
        self.stop()
        for child in self.children:
            child.disabled = True
        await interaction.message.edit(view=self)
        await interaction.response.send_message("❌ Trade declined.")


class TradeCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    @app_commands.command(name="trade", description="Propose a card trade with an
    @app_commands.describe(
        user="User to trade with",
        offer="Instance ID(s) you offer (comma-separated)",
        request="Instance ID(s) you want (comma-separated)"
    )
    async def trade(self, interaction: discord.Interaction, user: discord.Member,
        if user.id == interaction.user.id:
            await interaction.response.send_message(embed=embed_error("You can't
            return

        db.ensure_user(str(interaction.user.id), interaction.user.display_name)
        db.ensure_user(str(user.id), user.display_name)

        offer_ids = [x.strip() for x in offer.split(",")]
        request_ids = [x.strip() for x in request.split(",")]

        # Validate offer cards belong to initiator
        my_inv = db.get_user_inventory(str(interaction.user.id))
        my_instance_ids = [i["instance_id"] for i in my_inv]
        invalid_offer = []
        for oid in offer_ids:
            matches = [x for x in my_instance_ids if x.startswith(oid)]
            if not matches:
                invalid_offer.append(oid)

        if invalid_offer:
            await interaction.response.send_message(
                embed=embed_error(f"Cards not in your inventory: {', '.join(inval
            )
```

```
        return

    # Validate request cards belong to target
    their_inv = db.get_user_inventory(str(user.id))
    their_instance_ids = [i["instance_id"] for i in their_inv]
    invalid_req = []
    for rid in request_ids:
        matches = [x for x in their_instance_ids if x.startswith(rid)]
        if not matches:
            invalid_req.append(rid)

    if invalid_req:
        await interaction.response.send_message(
            embed=embed_error(f"{user.display_name} doesn't have those cards:
            ephemeral=True
        )
        return

    # Resolve full instance IDs
    def resolve_ids(id_list, inv_ids):
        resolved = []
        for short in id_list:
            matches = [x for x in inv_ids if x.startswith(short)]
            resolved.append(matches[0] if matches else short)
        return resolved

    full_offer = resolve_ids(offer_ids, my_instance_ids)
    full_request = resolve_ids(request_ids, their_instance_ids)

    trade_id = db.create_trade(str(interaction.user.id), str(user.id), full_o

    def card_list_str(ids, inv):
        parts = []
        for iid in ids:
            item = next((x for x in inv if x["instance_id"] == iid), None)
            if item:
                c = cu.get_card_by_id(item["card_id"])
                name = cu.get_card_display_name(c, item["variant"]) if c else
                parts.append(f"• {cu.get_variant_emoji(item['variant'])} {nam
        return "\n".join(parts) if parts else "None"

    embed = discord.Embed(
        title=f"🔄 Trade Proposal #{trade_id}",
        description=f"**{interaction.user.display_name}** → **{user.display_n
        color=0x3498DB
    )
    embed.add_field(name=f"📤 {interaction.user.display_name} offers", value=
```

```
        embed.add_field(name=f"📩 {interaction.user.display_name} requests", valu
        embed.set_footer(text="Trade expires in 2 minutes")

        view = TradeConfirmView(trade_id, user.id)
        await interaction.response.send_message(content=user.mention, embed=embed

    @app_commands.command(name="trade_history", description="View your recent tra
    async def trade_history(self, interaction: discord.Interaction):
        db.ensure_user(str(interaction.user.id), interaction.user.display_name)
        import sqlite3, os
        conn = db.get_conn()
        rows = conn.execute(
            "SELECT * FROM trades WHERE initiator_id = ? OR target_id = ? ORDER B
            (str(interaction.user.id), str(interaction.user.id))
        ).fetchall()
        conn.close()

        if not rows:
            await interaction.response.send_message(embed=embed_ok("📜 Trade Hist
            return

        embed = discord.Embed(title="📜 Your Trade History", color=0x3498DB)
        for row in rows:
            t = dict(row)
            status_emoji = {"completed": "✅", "declined": "❌", "pending": "⏳"
            other_id = t["target_id"] if t["initiator_id"] == str(interaction.use
            embed.add_field(
                name=f"{status_emoji} Trade #{t['trade_id']}",
                value=f"With: <@{other_id}> • {t['status'].capitalize()}\n{t['cre
                inline=True
            )
        await interaction.response.send_message(embed=embed, ephemeral=True)


# ────────────────────────────────────────────────────────────────────────────
# COG: Give (Gift)
# ────────────────────────────────────────────────────────────────────────────

class GiveCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    @app_commands.command(name="give", description="Give a card to another user")
    @app_commands.describe(user="Recipient", instance_id="Card instance ID (from
    async def give(self, interaction: discord.Interaction, user: discord.Member,
        if user.id == interaction.user.id:
            await interaction.response.send_message(embed=embed_error("You can't
```

```
            return

        db.ensure_user(str(interaction.user.id), interaction.user.display_name)
        db.ensure_user(str(user.id), user.display_name)

        my_inv = db.get_user_inventory(str(interaction.user.id))
        item = next((x for x in my_inv if x["instance_id"].startswith(instance_id

        if not item:
            await interaction.response.send_message(embed=embed_error("Card not f
            return

        card = cu.get_card_by_id(item["card_id"])
        if not card:
            await interaction.response.send_message(embed=embed_error("Invalid ca
            return

        if not card.get("giftable", True):
            await interaction.response.send_message(embed=embed_error(f"**{card['
            return

        db.transfer_card(item["instance_id"], str(user.id))

        v_emoji = cu.get_variant_emoji(item["variant"])
        embed = discord.Embed(
            title="🎁 Card Gifted!",
            description=(
                f"{v_emoji} **{cu.get_card_display_name(card, item['variant'])}**
                f"gifted to **{user.display_name}**!"
            ),
            color=0x00CC66
        )
        await interaction.response.send_message(embed=embed)


# ────────────────────────────────────────────────────────────────────────────
# COG: Card History
# ────────────────────────────────────────────────────────────────────────────

class HistoryCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    @app_commands.command(name="card_history", description="View market history a
    @app_commands.describe(name="Card name", variant="Variant (default: Standard)
    async def card_history(self, interaction: discord.Interaction, name: str, var
        card = cu.get_card_by_name(name) or cu.get_card_by_id(name.lower().replac
```

```
        if not card:
            await interaction.response.send_message(embed=embed_error(f"Card `{na
            return

        stats = db.get_card_history_stats(card["id"], variant)
        current_val = db.get_current_value(card["id"], variant, card["base_value"
        base_val = card["base_value"]

        pct_from_base = ((current_val - base_val) / base_val * 100) if base_val e
        pct_str = f"+{pct_from_base:.1f}%" if pct_from_base >= 0 else f"{pct_from

        v_emoji = cu.get_variant_emoji(variant)
        r_emoji = cu.get_rarity_emoji(card["rarity"])
        color = cu.get_rarity_color(card["rarity"])

        embed = discord.Embed(
            title=f"📊 {v_emoji} {card['name']} [{variant}] — History",
            color=color
        )
        embed.set_thumbnail(url=card["image_url"])
        embed.add_field(name="🌕 Base Value", value=f"{base_val:,}", inline=True)
        embed.add_field(name="📈 Current Value", value=f"{current_val:,} ({pct_st
        embed.add_field(name="📦 Total Sales", value=str(stats["total_sales"]), i
        embed.add_field(name="👥 Total Owners", value=str(stats["total_owners"]),
        if stats["total_sales"] > 0:
            embed.add_field(name="⬇️ Lowest Sale", value=f"{stats['min']:,}", inl
            embed.add_field(name="⬆️ Highest Sale", value=f"{stats['max']:,}", in
            embed.add_field(name="📊 Average Sale", value=f"{stats['avg']:,}", in

        if stats["daily"]:
            daily_str = ""
            for d in stats["daily"][:7]:
                daily_str += f"`{d['value_date']}` → avg {d['avg_price']:,} | sal
            embed.add_field(name="📅 Recent Daily History", value=daily_str or "N

        embed.set_footer(text="RFRXDex Market History")
        await interaction.response.send_message(embed=embed)


# ═══════════════════════════════════════════════════════════════════════════
# COG: Leaderboards
# ═══════════════════════════════════════════════════════════════════════════

class LeaderboardCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot
```

```
    @app_commands.command(name="leaderboard", description="View RFRXDex leaderboa
    @app_commands.describe(category="Category: collectors | rare | wealth")
    @app_commands.choices(category=[
        app_commands.Choice(name="📦 Collectors (most cards)", value="collectors"
        app_commands.Choice(name="💎 Rare (special variants)", value="rare"),
        app_commands.Choice(name="💰 Wealth (total value)", value="wealth"),
    ])
    async def leaderboard(self, interaction: discord.Interaction, category: str =
        if category == "collectors":
            rows = db.leaderboard_collectors()
            title = "📦 Top Collectors"
            field_name = "Cards"
            field_key = "card_count"
        elif category == "rare":
            rows = db.leaderboard_rare()
            title = "💎 Rare Card Leaders"
            field_name = "Special Cards"
            field_key = "rare_count"
        else:  # wealth
            rows = db.leaderboard_wealth()
            title = "💰 Wealthiest Collectors"
            field_name = "Total Value"
            field_key = "total_value"

        medals = ["🥇", "🥈", "🥉"]
        embed = discord.Embed(title=title, color=0xFFD700)

        if not rows:
            embed.description = "No data yet. Be the first to collect!"
        else:
            lines = []
            for i, row in enumerate(rows):
                medal = medals[i] if i < 3 else f"**#{i+1}**"
                val = row.get(field_key) or 0
                uid = row["user_id"]
                name = row.get("username") or f"User#{uid[-4:]}"
                lines.append(f"{medal} **{name}** — {field_name}: `{val:,}`")
            embed.description = "\n".join(lines)

        embed.set_footer(text="RFRXDex Leaderboard")
        await interaction.response.send_message(embed=embed)


# ────────────────────────────────────────────────────────────────────────────
# COG: Admin / Staff
# ────────────────────────────────────────────────────────────────────────────
```

```
class AdminCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot

    def is_admin(self, user: discord.Member) -> bool:
        if user.guild_permissions.administrator:
            return True
        return any(r.name in ["DEX Admin", "Staff", "Owner"] for r in getattr(use

    @app_commands.command(name="admin_give", description="[Admin] Give a card to
    @app_commands.describe(user="Target user", card_name="Card name", variant="Va
    async def admin_give(self, interaction: discord.Interaction, user: discord.Me
        if not self.is_admin(interaction.user):
            await interaction.response.send_message(embed=embed_error("Admin only
            return

        card = cu.get_card_by_name(card_name) or cu.get_card_by_id(card_name.lowe
        if not card:
            await interaction.response.send_message(embed=embed_error(f"Card `{ca
            return

        db.ensure_user(str(user.id), user.display_name)
        instance_id = cu.generate_instance_id()
        value = db.get_current_value(card["id"], variant, card["base_value"])
        db.add_card_to_inventory(str(user.id), card["id"], variant, value, instan

        if card.get("info_card_id"):
            db.grant_info_card(str(user.id), card["info_card_id"])

        await interaction.response.send_message(
            embed=embed_ok("✅ Card Given", f"Gave **{cu.get_card_display_name(ca
        )

    @app_commands.command(name="admin_spawn", description="[Admin] Force a card s
    async def admin_spawn(self, interaction: discord.Interaction):
        if not self.is_admin(interaction.user):
            await interaction.response.send_message(embed=embed_error("Admin only
            return
        await interaction.response.send_message("🌀 Forcing a spawn...", ephemera
        await interaction.client.spawn_system.do_spawn(channel_id=interaction.cha

    @app_commands.command(name="set_spawn_channel", description="[Admin] Add this
    async def set_spawn_channel(self, interaction: discord.Interaction):
        if not self.is_admin(interaction.user):
            await interaction.response.send_message(embed=embed_error("Admin only
            return
        cid = interaction.channel_id
```

```
        if cid not in interaction.client.spawn_system.spawn_channel_ids:
            interaction.client.spawn_system.spawn_channel_ids.append(cid)
            await interaction.response.send_message(embed=embed_ok("✅ Spawn Chan
        else:
            await interaction.response.send_message(embed=embed_ok("ℹ️ Already Ad

    @app_commands.command(name="coins", description="Check your coin balance")
    async def coins(self, interaction: discord.Interaction, user: Optional[discor
        target = user or interaction.user
        db.ensure_user(str(target.id), target.display_name)
        data = db.get_user(str(target.id))
        embed = discord.Embed(
            title=f"🪙 {target.display_name}'s Balance",
            description=f"**{data['coins']:,}** coins",
            color=0xF1C40F
        )
        await interaction.response.send_message(embed=embed)

    @app_commands.command(name="admin_coins", description="[Admin] Add or remove
    @app_commands.describe(user="Target user", amount="Amount (positive to add, n
    async def admin_coins(self, interaction: discord.Interaction, user: discord.M
        if not self.is_admin(interaction.user):
            await interaction.response.send_message(embed=embed_error("Admin only
            return
        db.ensure_user(str(user.id), user.display_name)
        db.add_coins(str(user.id), amount)
        data = db.get_user(str(user.id))
        await interaction.response.send_message(
            embed=embed_ok("✅ Coins Updated", f"**{user.display_name}** now has
        )



# ────────────────────────────────────────────────────────────────────────────
# COG: Help & Guide
# ────────────────────────────────────────────────────────────────────────────

GUIDE_PAGES = [
    {
        "title": "📖 RFRXDex Guide — Page 1: Getting Started",
        "color": 0x5865F2,
        "description": (
            "Welcome to **RFRXDex** — the official RFRX League collectible card b
            "Cards spawn automatically in designated channels every **5-15 minute
            "When one appears, hit the **✋ Sign** button before anyone else to c
            "Each card you catch is saved permanently to your collection. "
            "Info cards are granted **automatically** when you catch the parent c
```

```
        ),
        "fields": [
            ("🌀 How Spawns Work", (
                "• A card embed appears with an image + caption\n"
                "• Click **🖐 Sign** to catch it — first click wins\n"
                "• Once caught, the button locks for everyone\n"
                "• Cards spawn every **5-15 min** (randomised)\n"
                "• Only **tracks**, **team cards**, and **driver cards** spawn\n"
                "• Info cards are auto-granted — they never spawn on their own"
            ), False),
            ("📋 Catch Message Format", (
                "When you catch a card you'll see:\n"
                "```\n<User> caught ✨ McLaren [Shiny]! (#a3f2b1c8, +12.5%/+45.0%
                "`#ID` = unique card instance ID\n"
                "`+X%` = % change since last sale price\n"
                "`+Y%` = % change since base value"
            ), False),
        ]
    },
    {
        "title": "📖 RFRXDex Guide — Page 2: Rarities & Variants",
        "color": 0xFFD700,
        "description": "Every card has a **base rarity** and may also roll a **sp
        "fields": [
            ("⭐ Base Rarities", (
                "⚪ **Common** — Spawns most frequently\n"
                "🔵 **Rare** — Uncommon, solid value\n"
                "🟣 **Epic** — Hard to find, high value\n"
                "🟠 **Mythic** — Very rare, premium cards\n"
                "🌟 **Champion** — Extremely rare league legends\n"
                "🔴 **Limited** — Near-impossible, ultra collectible"
            ), True),
            ("✨ Special Variants & Chances", (
                "🏎 **GP Specs** — 20% · Active GP only *(China Special active)*\
                "🏆 **DOTD** — 15% · Driver of the Day\n"
                "✨ **Shiny** — 10% · Glowing version\n"
                "🔮 **Secret Rare** — 3% · Hidden gem\n"
                "💎 **Ultra Rare** — 1% · Top tier\n"
                "👑 **Collectors Special** — Collection reward"
            ), True),
            ("💰 Variant Value Multipliers", (
                "🏎 GP Specs → **×2.5**\n"
                "🏆 DOTD → **×1.8**\n"
                "✨ Shiny → **×2.0**\n"
                "🔮 Secret Rare → **×4.0**\n"
                "💎 Ultra Rare → **×8.0**\n"
                "👑 Collectors Special → **×15.0**"
```

```
        ), False),
    ]
},
{
    "title": "📖 RFRXDex Guide — Page 3: Your Collection",
    "color": 0x9B59B6,
    "description": "Manage and browse everything you\u2019ve collected.",
    "fields": [
        ("`/collection`", (
            "Shows all cards in your inventory:\n"
            "• Card name, variant & rarity\n"
            "• Current market value\n"
            "• Instance ID (used for selling/trading/gifting)\n"
            "• Your info cards section\n\n"
            "**Optional:** `/collection page:2` or `/collection user:@someone
        ), False),
        ("`/card <name>`", (
            "Full details on any card:\n"
            "• Image, type, rarity, current value\n"
            "• Marketable / giftable status\n"
            "• Available variants & linked info card\n\n"
            "*Example:* `/card McLaren` or `/card Red Bull`"
        ), True),
        ("`/coins`", (
            "Check your coin balance.\n"
            "You start with **500 coins**.\n"
            "Earn more by selling cards on the market.\n\n"
            "*Example:* `/coins` or `/coins user:@someone`"
        ), True),
    ]
},
{
    "title": "📖 RFRXDex Guide — Page 4: The Market",
    "color": 0xF1C40F,
    "description": "The **Global Market** lets anyone buy and sell cards. Pri
    "fields": [
        ("`/market`", (
            "Browse all active listings.\n"
            "• Card name, variant, rarity, price, seller\n"
            "• Filter by card: `/market card:McLaren`"
        ), True),
        ("`/sell <instance_id> <price>`", (
            "List one of your cards for sale.\n"
            "• Use the 8-char ID from `/collection`\n"
            "• Card leaves inventory until sold or delisted\n\n"
            "*Example:* `/sell a3f2b1c8 750`"
        ), True),
```

```
        ("`/buy <listing_id>`", (
            "Purchase a listing from the market.\n"
            "• You need enough coins\n"
            "• Coins go to seller instantly\n\n"
            "*Example:* `/buy 12`"
        ), True),
        ("`/delist <listing_id>`", (
            "Cancel your own listing.\n"
            "• Card is returned to your inventory\n"
            "• No fee for cancelling\n\n"
            "*Example:* `/delist 12`"
        ), True),
        ("💡 Dynamic Pricing", (
            "`new_value = 70% × old + 30% × sale_price` + ±2% drift\n"
            "High demand = rising prices. Low activity = gradual decrease."
        ), False),
    ]
},
{
    "title": "📖 RFRXDex Guide — Page 5: Trading & Gifting",
    "color": 0x3498DB,
    "description": "Trade cards directly with other users, or gift them for f
    "fields": [
        ("`/trade @user <offer> <request>`", (
            "Propose a card trade with another user.\n"
            "• `offer` = your instance ID(s), comma-separated\n"
            "• `request` = their instance ID(s), comma-separated\n"
            "• They get a prompt with ✅ Accept / ❌ Decline\n"
            "• Trade expires after **2 minutes**\n\n"
            "*Example:* `/trade @User a3f2b1c8 9d4e2f1a`"
        ), False),
        ("`/trade_history`", (
            "View your last 10 trades:\n"
            "✅ Completed • ❌ Declined • ⏳ Pending"
        ), True),
        ("`/give @user <instance_id>`", (
            "Gift a card for free — no coins involved.\n"
            "• Card must be **giftable**\n"
            "• Cannot gift to yourself\n\n"
            "*Example:* `/give @User a3f2b1c8`"
        ), True),
        ("💡 Finding Instance IDs", (
            "Use `/collection` to see your cards.\n"
            "Each shows a short ID like `#a3f2b1c`.\n"
            "You only need the **first few characters** — the bot finds the f
        ), False),
    ]
```

```
},
{
    "title": "📖 RFRXDex Guide — Page 6: Stats & Leaderboards",
    "color": 0x00CC66,
    "description": "Track card prices over time and compete with other collec
    "fields": [
        ("`/card_history <name>`", (
            "Full market history for any card:\n"
            "• Base value vs current value + % change\n"
            "• Total sales & unique owners\n"
            "• Lowest / Highest / Average sale price\n"
            "• Last 7 days of daily price data\n\n"
            "*Example:* `/card_history McLaren` or with `variant:Shiny`"
        ), False),
        ("`/leaderboard`", (
            "Top collectors in three categories:\n\n"
            "📦 **Collectors** — most cards owned\n"
            "💎 **Rare** — most special variant cards\n"
            "💰 **Wealth** — highest total card value\n\n"
            "*Example:* `/leaderboard category:wealth`"
        ), False),
    ]
},
{
    "title": "📖 RFRXDex Guide — Page 7: Admin Commands",
    "color": 0xFF6B35,
    "description": "Staff-only commands. Requires **Administrator** permissio
    "fields": [
        ("`/set_spawn_channel`", "Add current channel to spawn rotation.", Tr
        ("`/admin_spawn`", "Force an immediate card spawn here. Great for tes
        ("`/admin_give @user <card> [variant]`", (
            "Give any card directly to a user.\n"
            "Info card auto-granted if applicable.\n"
            "*Example:* `/admin_give @User Red Bull variant:Shiny`"
        ), False),
        ("`/admin_coins @user <amount>`", (
            "Add or remove coins. Use negative to deduct.\n"
            "*Example:* `/admin_coins @User 1000` or `/admin_coins @User -200
        ), False),
        ("⚙️ Config Tips", (
            "• Edit `cards.json` to add/modify cards\n"
            "• Set `gp_specs.active` to the current active GP spec ID\n"
            "• Set `marketable: false` to block listing\n"
            "• Set `giftable: false` to block gifting"
        ), False),
    ]
},
```

```
]


class GuidePaginatorView(discord.ui.View):
    def __init__(self, pages: list, current_page: int = 0):
        super().__init__(timeout=120)
        self.pages = pages
        self.current_page = current_page
        self._update_buttons()

    def _update_buttons(self):
        self.prev_button.disabled = self.current_page == 0
        self.next_button.disabled = self.current_page == len(self.pages) - 1
        self.page_counter.label = f"{self.current_page + 1} / {len(self.pages)}"

    def build_embed(self) -> discord.Embed:
        page = self.pages[self.current_page]
        embed = discord.Embed(
            title=page["title"],
            description=page["description"],
            color=page["color"]
        )
        for field in page.get("fields", []):
            embed.add_field(name=field[0], value=field[1], inline=field[2])
        embed.set_footer(text=f"RFRXDex Guide • Page {self.current_page + 1}/{len
        return embed

    @discord.ui.button(label="◀", style=discord.ButtonStyle.secondary)
    async def prev_button(self, interaction: discord.Interaction, button: discord
        self.current_page = max(0, self.current_page - 1)
        self._update_buttons()
        await interaction.response.edit_message(embed=self.build_embed(), view=se

    @discord.ui.button(label="1 / 7", style=discord.ButtonStyle.primary, disabled
    async def page_counter(self, interaction: discord.Interaction, button: discor
        pass

    @discord.ui.button(label="▶", style=discord.ButtonStyle.secondary)
    async def next_button(self, interaction: discord.Interaction, button: discord
        self.current_page = min(len(self.pages) - 1, self.current_page + 1)
        self._update_buttons()
        await interaction.response.edit_message(embed=self.build_embed(), view=se


class HelpCog(commands.Cog):
    def __init__(self, bot):
        self.bot = bot
```

```
@app_commands.command(name="help", description="View all RFRXDex commands at
async def help(self, interaction: discord.Interaction):
    embed = discord.Embed(
        title="📖 RFRXDex — Command Reference",
        description=(
            "The **RFRX League** collectible card bot 🏎️\n"
            "Spawn, catch, trade, and collect rare cards!\n\n"
            "💡 *For a full walkthrough, use `/guide`*"
        ),
        color=0x5865F2
    )
    embed.add_field(name="🗂️ Collection", value=(
        "`/collection` — View your cards\n"
        "`/card <n>` — Card details & image\n"
        "`/coins` — Check coin balance"
    ), inline=True)
    embed.add_field(name="🏪 Market", value=(
        "`/market` — Browse all listings\n"
        "`/sell <id> <price>` — List a card\n"
        "`/buy <listing_id>` — Buy a card\n"
        "`/delist <id>` — Cancel your listing"
    ), inline=True)
    embed.add_field(name="🔄 Trading & Gifts", value=(
        "`/trade @user <offer> <want>` — Propose trade\n"
        "`/trade_history` — Past trades\n"
        "`/give @user <id>` — Gift a card"
    ), inline=True)
    embed.add_field(name="📊 Stats", value=(
        "`/card_history <n>` — Price history\n"
        "`/leaderboard` — Top collectors"
    ), inline=True)
    embed.add_field(name="⚙️ Admin", value=(
        "`/admin_give` • `/admin_spawn`\n"
        "`/set_spawn_channel` • `/admin_coins`"
    ), inline=True)
    embed.add_field(name="✨ Variants", value=(
        "🏎️ GP Specs (20%) • 🏆 DOTD (15%)\n"
        "✨ Shiny (10%) • 🔮 Secret Rare (3%)\n"
        "💎 Ultra Rare (1%) • 👑 Collectors Special"
    ), inline=True)
    embed.add_field(name="⭐ Rarities", value=(
        "⚪ Common → 🔵 Rare → 🟣 Epic → 🟠 Mythic → 🌟 Champion → 🔴 Limite
    ), inline=False)
    embed.set_footer(text="RFRXDex • Use /guide for a full walkthrough")
    await interaction.response.send_message(embed=embed, ephemeral=True)
```

```
    @app_commands.command(name="guide", description="Full interactive RFRXDex gui
    @app_commands.describe(page="Start on a specific page (1-7)")
    async def guide(self, interaction: discord.Interaction, page: int = 1):
        page_idx = max(1, min(page, len(GUIDE_PAGES))) - 1
        view = GuidePaginatorView(GUIDE_PAGES, current_page=page_idx)
        embed = view.build_embed()
        await interaction.response.send_message(embed=embed, view=view, ephemeral


# ────────────────────────────────────────────────────────────────────────────
# Setup function
# ────────────────────────────────────────────────────────────────────────────

async def setup(bot):
    await bot.add_cog(CollectionCog(bot))
    await bot.add_cog(MarketCog(bot))
    await bot.add_cog(TradeCog(bot))
    await bot.add_cog(GiveCog(bot))
    await bot.add_cog(HistoryCog(bot))
    await bot.add_cog(LeaderboardCog(bot))
    await bot.add_cog(AdminCog(bot))
    await bot.add_cog(HelpCog(bot))
```